Introduction

A decorator in Python is a function that modifies the behavior of another function or method.

Decorators allow for reusable modifications without changing the original function's code, making them a powerful tool for code organization and reuse.

Understanding Decorators

Decorators are commonly used to:

1. **Modify functions dynamically** - Extend or alter functionality without modifying the function itself.
2. **Improve code readability** - Make code more concise and maintainable.
3. **Implement cross-cutting concerns** - Such as logging, authentication, and caching.

Basic Syntax of a Decorator

A decorator is applied to a function using the `@decorator_name` syntax.

```python
def my_decorator(func):
 def wrapper():
 print("Something before the function runs")
 func()
 print("Something after the function runs")
 return wrapper
@my_decorator
def say_hello():
 print("Hello!")
say_hello()
```

Output:

```
```

Something before the function runs

Hello!

Something after the function runs

```

### Using Arguments in Decorators

Decorators can handle arguments by using `*args` and `**kwargs` in the wrapper function.

```python
def repeat_decorator(func):
 def wrapper(*args, **kwargs):
 print("Executing function...")
 result = func(*args, **kwargs)
 print("Execution complete.")
 return result
 return wrapper
@repeat_decorator
def greet(name):
 print(f"Hello, {name}!")
greet("Alice")
```

Output:

```
Executing function...
Hello, Alice!
Execution complete.
```

Built-in Decorators in Python

Python provides several built-in decorators, such as:

1. **`@staticmethod`** - Defines a static method within a class.
2. **`@classmethod`** - Defines a class method that receives the class as an implicit parameter.
```

3. **`@property`** - Allows defining a method as a property.

Example: Using `@staticmethod` and `@classmethod`

```python
class MathOperations:
 @staticmethod
 def add(x, y):
 return x + y

 @classmethod
 def multiply(cls, x, y):
 return x * y
print(MathOperations.add(3, 4)) # Output: 7
print(MathOperations.multiply(3, 4)) # Output: 12
```

Chaining Multiple Decorators

Multiple decorators can be applied to a function.

```python
def decorator1(func):
 def wrapper():
 print("Decorator 1")
 func()
 return wrapper
def decorator2(func):
 def wrapper():
 print("Decorator 2")
 func()
 return wrapper
@decorator1
@decorator2
def say_hi():
 print("Hi!")
```

say_hi()

```

**Output:**

```
Decorator 1
Decorator 2
Hi!
```

### Conclusion

- Decorators allow modifying function behavior without altering their code.
- They improve code readability and maintainability.
- Python includes built-in decorators like `@staticmethod`, `@classmethod`, and `@property`.
- Multiple decorators can be applied to a function for layered functionality.
```